



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Análisis de Componentes Principales (PCA)

EIN092B - Visualización

Jorge Portilla

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Tabla de Contenidos

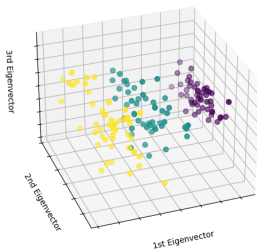
- 1** Introducción
- 2** Conceptos básicos
- 3** Análisis de componentes principales
- 4** Ejemplo paso a paso para calcular PCA
- 5** Aplicación de PCA
- 6** t-SNE
- 7** Representaciones 2D y 3D de algunos dataset públicos

Introducción

- **“Curse of Dimensionality”**: Concepto que aborda los desafíos emergentes al trabajar con conjuntos de datos de alta dimensionalidad.
- Problemas asociados:
 - Dispersión de datos.
 - Difícil interpretación y visualización de los datos.
 - Mayor complejidad computacional.
- Enfoque principal: ¿Cómo determinar el número óptimo de dimensiones para nuestro dataset?
- Estrategias de Reducción de Dimensionalidad:
 - Análisis de Componentes Principales (PCA).
 - Análisis Discriminante Lineal (LDA).
 - T-Distributed Stochastic Neighbor Embedding (t-SNE).
 - Otras técnicas relevantes.

Objetivos de PCA

First three PCA dimensions



- PCA: Herramienta para análisis de datos en alta dimensión.
- Reduce dimensiones manteniendo información relevante.
- **Aplicaciones:**
 - Compresión de datos sin pérdida significativa.
 - Eliminación de redundancias en imágenes y selección de características.
 - Descripción de regiones independiente de la rotación.

Importancia y aplicaciones en diversos campos...

¹Imagen tomada de <https://scikit-learn.org/stable/autoexamples/datasets/irisdataset>

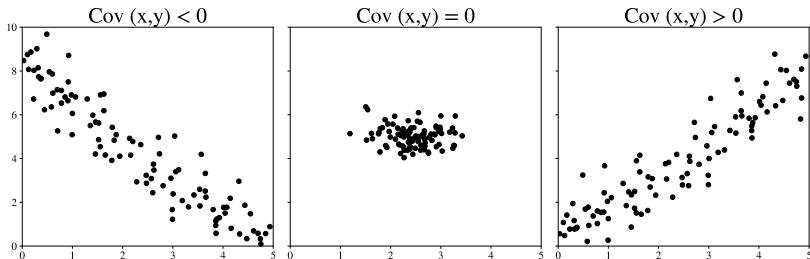
Conceptos básicos

- **Varianza:** medida que indica qué tan dispersos están los valores con respecto a la media.

$$\text{var}(x) = \sigma_x^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_x)^2$$

- **Covarianza:** representa la tasa de cambio entre dos variables aleatorias.

$$\text{Cov}(x, y) = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_x)(y_i - \mu_y)$$



Matriz de covarianza

Sea el vector $\mathbf{x} = \{x_1, \dots, x_N\}$, un vector de variables aleatorias.

$$\mathbf{C}_x = \mathbf{C}_x^{N \times N} = (c_{ij} | c_{ij} = \text{Cov}(x_i, x_j))$$

Teniendo en cuenta que $\text{Cov}(x, x) = \sigma_x^2$ y sea $\mathbf{x} = \{x, y, z\}$:

$$\mathbf{C}_x = \begin{pmatrix} \sigma_x^2 & \text{Cov}(x, y) & \text{Cov}(x, z) \\ \text{Cov}(x, y) & \sigma_y^2 & \text{Cov}(y, z) \\ \text{Cov}(x, z) & \text{Cov}(y, z) & \sigma_z^2 \end{pmatrix}$$

- $\mathbf{C}_x$ es una matriz simétrica.
- La matriz de covarianza también se puede definir como:

$$\mathbf{C}_x = E[(\mathbf{x} - \mu_x)(\mathbf{x} - \mu_x)^T] = \frac{1}{n} \sum_{k=1}^n (\mathbf{x}_k \mathbf{x}_k^T - \mu_x \mu_x^T)$$

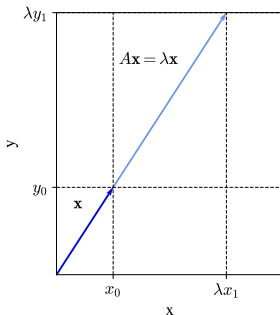
Vectores y valores propios

Valores y vectores propios

Dada una transformación lineal A y un vector $\mathbf{x}$ distinto de cero, $\mathbf{x}$ es un vector propio de esta transformación si cumple la siguiente expresión:

$$A\mathbf{x} = \lambda\mathbf{x}, \quad (1)$$

donde λ es un escalar. Donde λ es un valor propio de A correspondiente al vector propio $\mathbf{x}$.



Vectores y valores propios

- Ejemplos

- Vector no propio:

$$A\mathbf{x}_1 = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

- Vector propio:

$$A\mathbf{x}_1 = \begin{bmatrix} 3 & 1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix} = 4 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \lambda \mathbf{x}_1$$

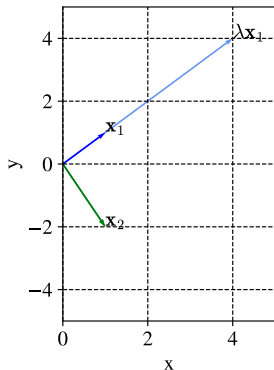
- La matriz cuadrada se puede considerar como una matriz de transformación.
- Si se multiplica la matriz de transformación por un vector, se obtiene otro vector llevado al espacio de la transformación.

Valores propios

Para el ejemplo anterior:

$$A\mathbf{x}_1 = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} -0,85 \\ 0,52 \end{bmatrix} = 3,61 \begin{bmatrix} -0,85 \\ 0,52 \end{bmatrix} = \lambda \mathbf{x}_1$$

$$A\mathbf{x}_2 = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 0,52 \\ -0,85 \end{bmatrix} = 1,36 \begin{bmatrix} 0,52 \\ -0,85 \end{bmatrix} = \lambda \mathbf{x}_2$$



- Los valores y vectores propios siempre vienen en pares.
- Propiedades de los vectores y valores propios:
 - Sólo para matrices cuadradas.
 - Filas linealmente independientes (relacionado al rango de la matriz).
 - Los vectores propios para una matriz de transformación son ortogonales entre sí (si la matriz es simétrica).

Vectores y valores propios

- Los vectores propios representan los ejes del nuevo espacio que lleva la transformación.
- Los vectores propios se normalizan para generar un espacio ortogonal.
- Por lo general, obtener los vectores propios se vuelve costoso computacionalmente para matrices de alta dimensionalidad.
- Existen diversos métodos iterativos especializados para calcular los valores y vectores propios (método QR, el método de la potencia, entre otros). Algunos de estos métodos han sido implementados y optimizados para acelerar el cálculo en bibliotecas de informática científica, como NumPy y SciPy, así como en programas como MATLAB.

Análisis de componentes principales

PCA

Transformación lineal ortogonal que mapea los datos a un nuevo sistema coordinado tal que la mayor varianza de cualquier proyección en los datos se encuentra en la primera componente (PC1), la segunda mayor varianza en la segunda componente (PC2), y así sucesivamente.

- PCA transforma un vector $\mathbf{x} = [x_1, x_2, \dots, x_N]$ en un vector $\mathbf{y}$, así:

$$\mathbf{y} = A(\mathbf{x} - \mu_x),$$

donde μ_x corresponde al vector de medias, y A es la matriz de vectores propios en cada fila, ordenados de mayor a menor.

- Los vectores y valores propios son obtenidos de la matriz de covarianza $\mathbf{C}_x$. Donde $\mathbf{C}_x$ es simétrica y real, por lo que se asegura la existencia de N vectores ortogonales².

²Noble and Daniels, 1988

- Como los vectores de A son ortogonales, $A^{-1} = A^T$. Por lo que para recuperar $\mathbf{x}$ así:

$$\mathbf{x} = A^T \mathbf{y} + \mu_x$$

- Si se forma una matriz A_k ocupando solo los k vectores propios con mayores valores propios de $\mathbf{C}_x$, la transformación sería de orden $k \times N$.
- Los vectores $\mathbf{y}$ serían de longitud k , y la reconstrucción sería inexacta.

$$\hat{\mathbf{x}} = A_k^T \mathbf{y} + \mu_x,$$

- El error cuadrático medio (MSE) de $\hat{\mathbf{x}}$ y $\mathbf{x}$ está dado así:

$$\text{MSE} = \sum_{j=1}^n \lambda_j - \sum_{j=1}^k \lambda_j,$$

donde $\lambda = \lambda_1, \dots, \lambda_n$ corresponde al conjunto de valores propios de mayor a menor.

Ejemplo paso a paso para calcular PCA

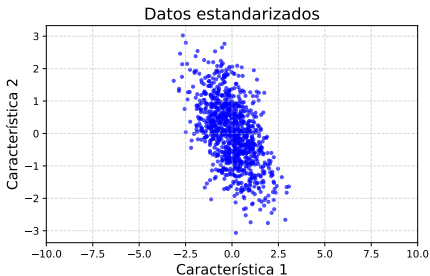
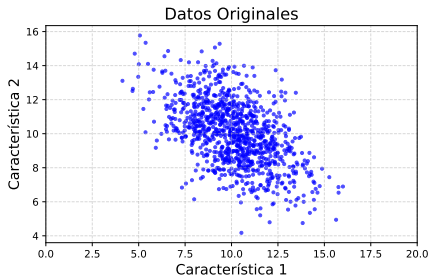
Pasos para calcular PCA

Crear un conjunto de datos: Generar muestras aleatorias de una distribución normal multivariante usando $\mathbf{C}_{x1}$ y μ_{x1} definidos así:

$$\mu_{x1} = \begin{bmatrix} 10 & 10 & 10 \end{bmatrix} \quad \mathbf{C}_{x1} = \begin{bmatrix} 3,5 & -1,6 & -1,6 \\ -1,6 & 3,5 & -1,6 \\ -1,6 & -1,6 & 3,5 \end{bmatrix}$$

Se generan 1000 muestras con una dimensión de 1000×3 .

1. Restar la media o estandarizar: Al restar μ_x a los datos se centra el espacio en las medias, por lo que la media de los datos estandarizados es cero ($\mu_x = E[x] = 0$)



Pasos para calcular PCA

2. Calcular la matriz de covarianza

$$\mathbf{C}_x = \begin{bmatrix} 1,0 & -0,47 & -0,42 \\ -0,47 & 1,0 & -0,48 \\ -0,42 & -0,48 & 1,0 \end{bmatrix}$$

3. Calcular los valores y vectores propios de la matriz de covarianza:

$$eigval = \begin{bmatrix} 5,1 & 0,3 & 5,1 \end{bmatrix} \quad eigvec = \begin{bmatrix} -0,81 & -0,57 & -0,05 \\ -0,40 & 0,57 & -0,67 \\ -0,40 & 0,57 & 0,73 \end{bmatrix}$$

- Ordenar los vectores propios en orden descendente según el valor propio. El valor propio más grande corresponde a la componente principal número 1 (PC1).
- El número de vectores propios es igual al número de características.
- Cada vector propio representa una dirección de variabilidad.

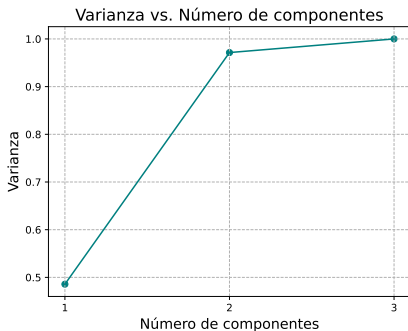
$$eigval = \begin{bmatrix} 5,1 & 5,1 & 0,3 \end{bmatrix} \quad eigvec = \begin{bmatrix} -0,05 & -0,81 & -0,57 \\ -0,67 & -0,40 & 0,57 \\ 0,73 & -0,40 & 0,57 \end{bmatrix}$$

Pasos para calcular PCA

4. Selección de número de componentes y calculo de la varianza:

- Inspeccionar la varianza contenida en cada característica.

$$varianza = \begin{bmatrix} 0,48 & 0,48 & 0,02 \end{bmatrix}$$

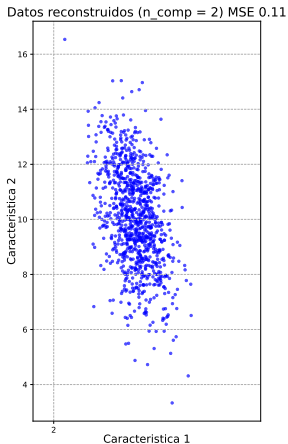
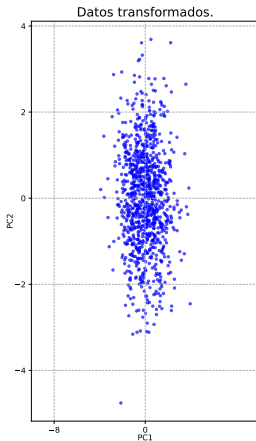
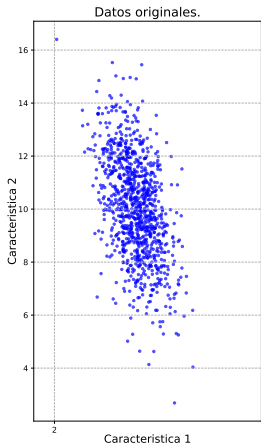


- Selección del número de componentes, en este caso nos quedamos con 2 (selecccionar los dos primeros vectores propios).

$$eigvec = \begin{bmatrix} -0,05 & -0,81 \\ -0,67 & -0,40 \\ 0,73 & -0,40 \end{bmatrix}$$

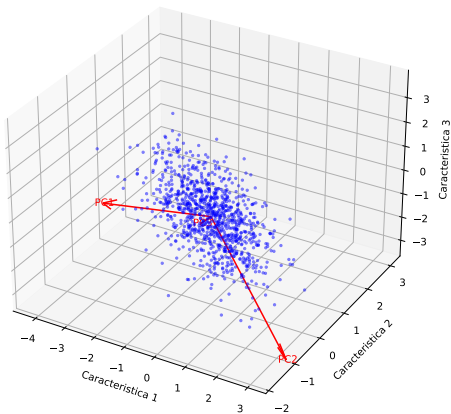
Pasos para calcular PCA

5. Transformar los datos usando los vectores propios: Producto punto entre los vectores propios con los datos originales.
6. Recuperar datos originales a partir de la representación obtenida:



Pasos para calcular PCA

Datos después de restar la media



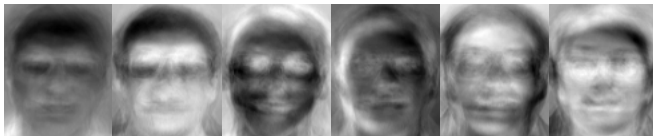
Aplicación de PCA

Aplicación clásica - Eigenfaces

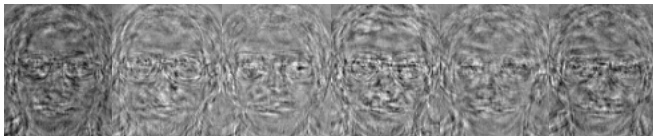


- Aplicación por defecto en el estado del arte.
- Dataset ATT de 400 imágenes de rostros con dimensión de 128×128 .

Eigenfaces

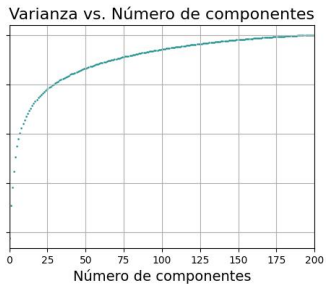
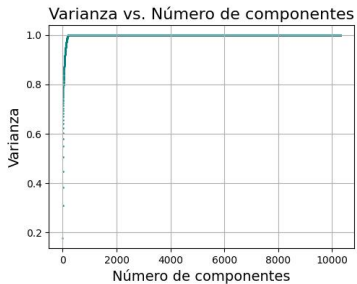


- Eigenfaces con valores propios más altos (de 200). Los vectores propios de la matriz de covarianza del conjunto de datos tienen apariencia de rostros borrosos y reciben el nombre de Eigenfaces.
- La mayor parte de la información del conjunto de datos está contenida en las Eigenfaces con mayores valores propios asociados.



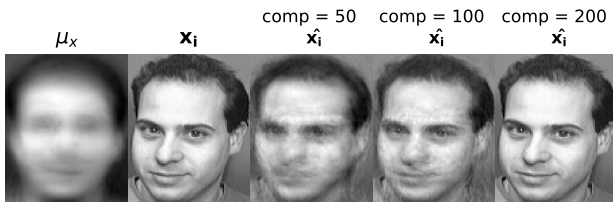
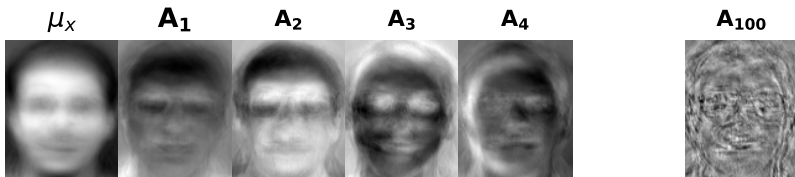
- Eigenfaces con valores propios mayores a 0 más bajos (de 200). Las Eigenfaces asociadas a los menores valores propios pueden considerarse como ruido desde el punto de vista de clasificación.

Eigenfaces



- Utilizando pocas componentes se pueden obtener buenos resultados en problemas de clasificación.

Eigenfaces



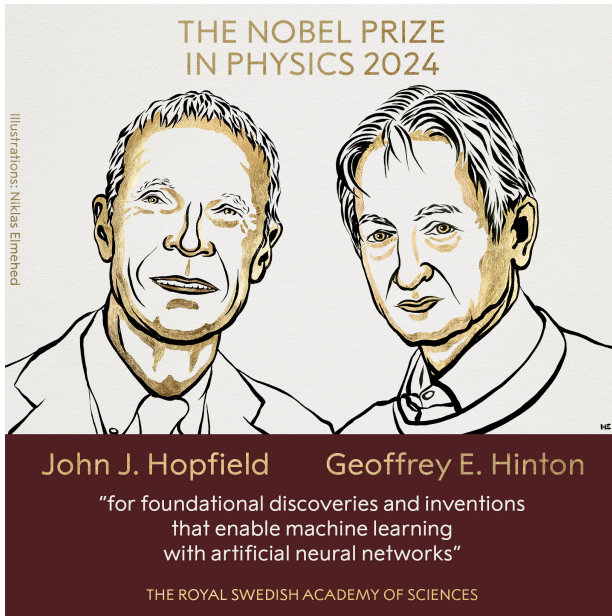
Conclusiones PCA

- PCA es una transformación lineal ortogonal catalogado como una técnica de aprendizaje no supervisado.
- Podemos reconstruir los datos originales a partir de los datos transformados.
- Algoritmo Hebbiano Generalizado (GHA), es un método adaptativo para hacer PCA sin calcular los valores y vectores propios. Aprende por ejemplo, 8 componentes para no calcular las otras.
- Características obtenidas con PCA se pueden usar para alimentar algoritmos y modelos para resolver problemas de regresión y clasificación.

t-SNE



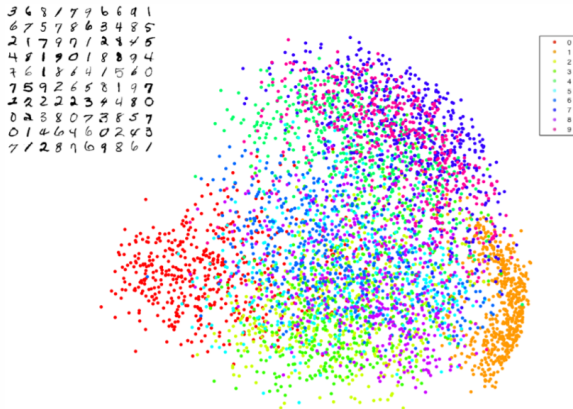
t-distributed stochastic neighbor embedding (t-SNE)



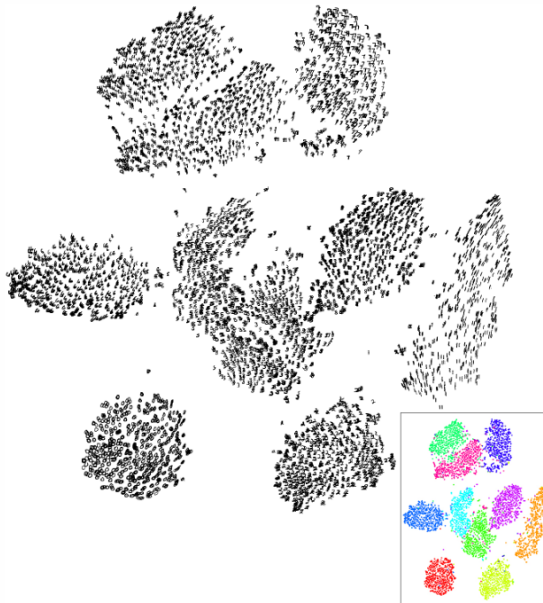
t-distributed stochastic neighbor embedding (t-SNE)

- Fue desarrollado por Laurens van der Maaten y Geoffrey Hinton en 2008.
- t-SNE es una **herramienta para visualización y exploración de datos de alta dimensionalidad**.
- t-SNE intenta conservar la estructura local — La vecindad de baja dimensión debe ser la misma que la vecindad original (en PCA puede no suceder lo mismo).
- **A diferencia de PCA, t-SNE permite separar datos que no pueden ser separados linealmente.**
- A diferencia del PCA, t-SNE casi sólo se utiliza para la visualización. **No es fácil integrar nuevos puntos de datos.**
- En el siguiente [enlace](#) puede probar t-SNE para distintos datos y configuraciones de parámetros.

PCA 2D para el dataset MNIST



t-SNE 2D para el dataset MNIST



t-distributed stochastic neighbor embedding (t-SNE)

- **Stochastic:** La función objetivo no es convexa, lo que lleva a resultados variables según la inicialización.
- **Neighbor Embedding:** Mapear los puntos del espacio original de alta dimensión en el espacio de baja dimensión correspondiente, preservando al máximo la estructura de vecindad de los datos.
- Pasos generales de SNE⁴:
 - Calcular la probabilidad de que un punto considere a otro como vecino en el espacio original de alta dimensión.
 - Colocar los puntos en un espacio reducido (2D o 3D) y definir probabilidades de vecindad allí también.
 - Comparar ambas distribuciones de vecindades: los puntos cercanos deben seguir siéndolo, y los lejanos deben mantenerse separados.
 - Ajustar iterativamente las posiciones en el espacio reducido: los puntos cercanos se atraen y los lejanos se repelen.

⁴Los detalles del funcionamiento del t-SNE se encuentran en <https://jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf>. Para nuestro curso, no es necesario entrar en detalles sobre la demostración matemática.

Idea básica de SNE:

- **Codificar** la información de vecindad de alta dimensión como una distribución.
- **Intuición:** Recorrido aleatorio entre puntos de datos.
 - Alta probabilidad de saltar o conectar a un punto cercano.
- Encontrar puntos de baja dimensión cuya distribución de vecindad sea similar.
- t-SNE calcula la distancia euclidiana entre cada par de puntos en el espacio original de alta dimensión. Luego, basado en estas distancias, construye una distribución de probabilidad que mide la similitud entre los puntos⁵.
- ¿Cómo se mide la distancia entre distribuciones?
 - Medida más común: divergencia *Kullback-Leibler* (KL)⁶

⁵Puntos más cercanos tendrán una probabilidad más alta de ser considerados vecinos.

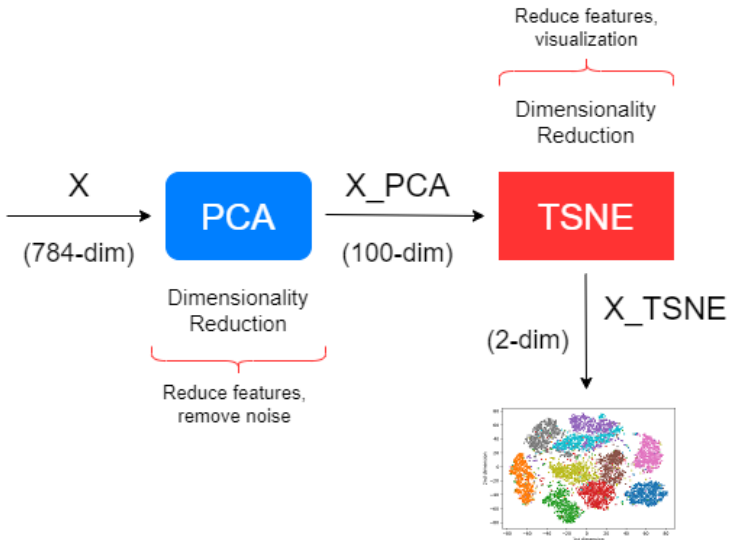
⁶KL es una medida utilizada para cuantificar la diferencia entre dos distribuciones de probabilidad. En t-SNE, KL se emplea como una función de costo para optimizar la representación de los datos en un espacio de menor dimensión. El objetivo es minimizar la KL entre la distribución de los datos de alta dimensionalidad y la correspondiente representación en baja dimensión.

Intuición de t-SNE

- El objetivo es tomar un conjunto de datos en un espacio de alta dimensión y encontrar una representación de esos puntos en un espacio de dimensión inferior, normalmente el plano 2D.
- El algoritmo **no es lineal** y se adapta a los datos subyacentes, realizando diferentes transformaciones en distintas regiones. Esas diferencias pueden ser una fuente importante de confusión.
- t-SNE posee un parámetro ajustable, *perplexity*, que indica un equilibrio entre los aspectos locales y globales de los datos. El parámetro es, **una suposición sobre el número de vecinos cercanos que tiene cada punto**. El parámetro *perplexity* se puede interpretar como un trade-off entre preservar la estructura local o global en la representación reducida obtenida con t-SNE⁷.

⁷Elegir un valor adecuado de *perplexity* permite que t-SNE represente los datos de manera que tanto la estructura local (pequeños grupos de puntos) como la global (relación entre clústeres) se vean reflejadas en la reducción de dimensionalidad.

PCA + t-SNE



Hiperparámetros de t-SNE

- El rendimiento de SNE es bastante robusto a los cambios en la perplexity, y los **valores típicos están en el rango entre 5 y 50** (valores sugeridos en el paper original ⁸).
- Perplexity ajusta el ancho de la distribución gaussiana utilizada para calcular la similitud de cada punto con sus vecinos.
- Un valor bajo de perplexity (por ejemplo, 5-10) permite que el algoritmo se enfoque en vecinos muy cercanos y detecte relaciones en los patrones locales. Por el contrario, un valor de perplexity alto (por ejemplo, 30-50) permite abarcar un mayor rango de vecinos, lo que ayuda a capturar estructuras de mayor escala en los datos.

⁸<https://jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf>

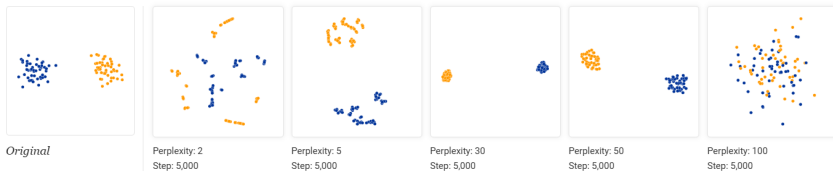
Desventajas de t-SNE

- **t-SNE no es determinista**, es decir, no siempre produce resultados similares en ejecuciones sucesivas. Además, requiere la sintonización de hiperparámetros relacionados con el proceso de optimización.
- Como resultado, t-SNE se usa principalmente para visualizar datos y no para alimentar modelos de aprendizaje automático.

Hiperparámetros de t-SNE

- Perplexity
 - Equilibra relaciones locales y globales. Aprox. número de vecinos cercanos.
 - Valores recomendados por los autores en el rango de 5 a 50.
- Tasa de aprendizaje
 - Tamaño de los pasos de optimización. Un valor bajo, puede llevar a una convergencia lenta.
 - Valores recomendados 10 a 1000.
- Número de iteraciones
 - Por lo general, entre 250 y 5000 iteraciones.
- Dimensionalidad de salida
 - Espacio objetivo de proyección (2D o 3D).

Ejemplo de t-SNE



- Para asegurar el correcto funcionamiento del algoritmo, perplexity debe ser menor que el número de puntos. En caso contrario, las implementaciones pueden tener un comportamiento inesperado.
- Para valores de perplexity fuera del rango recomendado por los autores, los gráficos tienen comportamientos inesperados.

Perplexity



La variación del parámetro de perplexity puede dar lugar a visualizaciones drásticamente distintas que muestren estructuras diferentes, como muestra la figura.

⁹<https://distill.pub/2016/misread-tsne/>

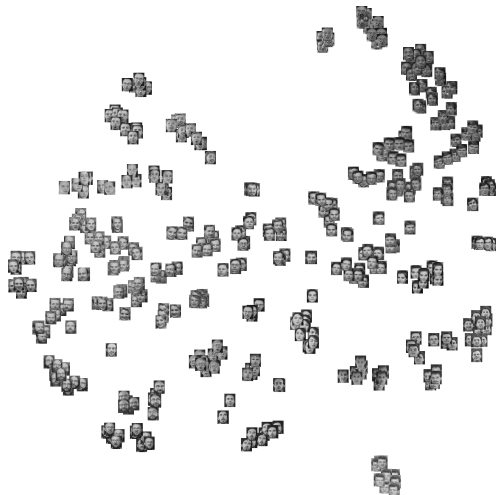
Limitaciones de t-sne

Limitaciones del t-SNE:

- **t-SNE no funciona bien para problemas de dimensionalidad general en los que la dimensión de la representación es superior a 2D o 3D.**
- **Maldición de la dimensionalidad (t-SNE emplea distancias euclidianas entre vecinos cercanos).**
- El número de perplexity, el número de iteraciones, entre otros hiperparámetros, deben elegirse manualmente.

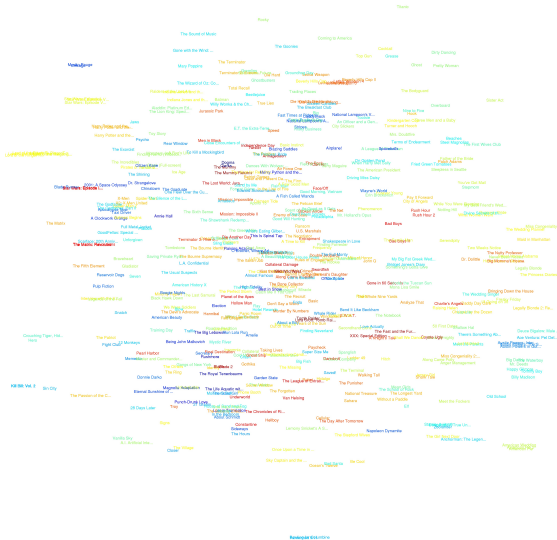
Representaciones 2D y 3D de algunos dataset públicos

Olivetti dataset



¹⁰https://lvdmaaten.github.io/tsne/examples/olivetti_tsne.jpg

Netflix dataset



¹¹https://lvdmaaten.github.io/tsne/examples/netflix_tsne.jpg

Profundizar en visualización de datos con t-SNE

- En el siguiente enlace se encuentra la página oficial de Laurens van der Maaten, que contiene detalles sobre la implementación de t-SNE en distintos lenguajes y ejemplos de su aplicación a diversos conjuntos de datos:
<https://lvdmaaten.github.io/tsne/>



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Análisis de Componentes Principales (PCA)

EIN092B - Visualización

Jorge Portilla

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile